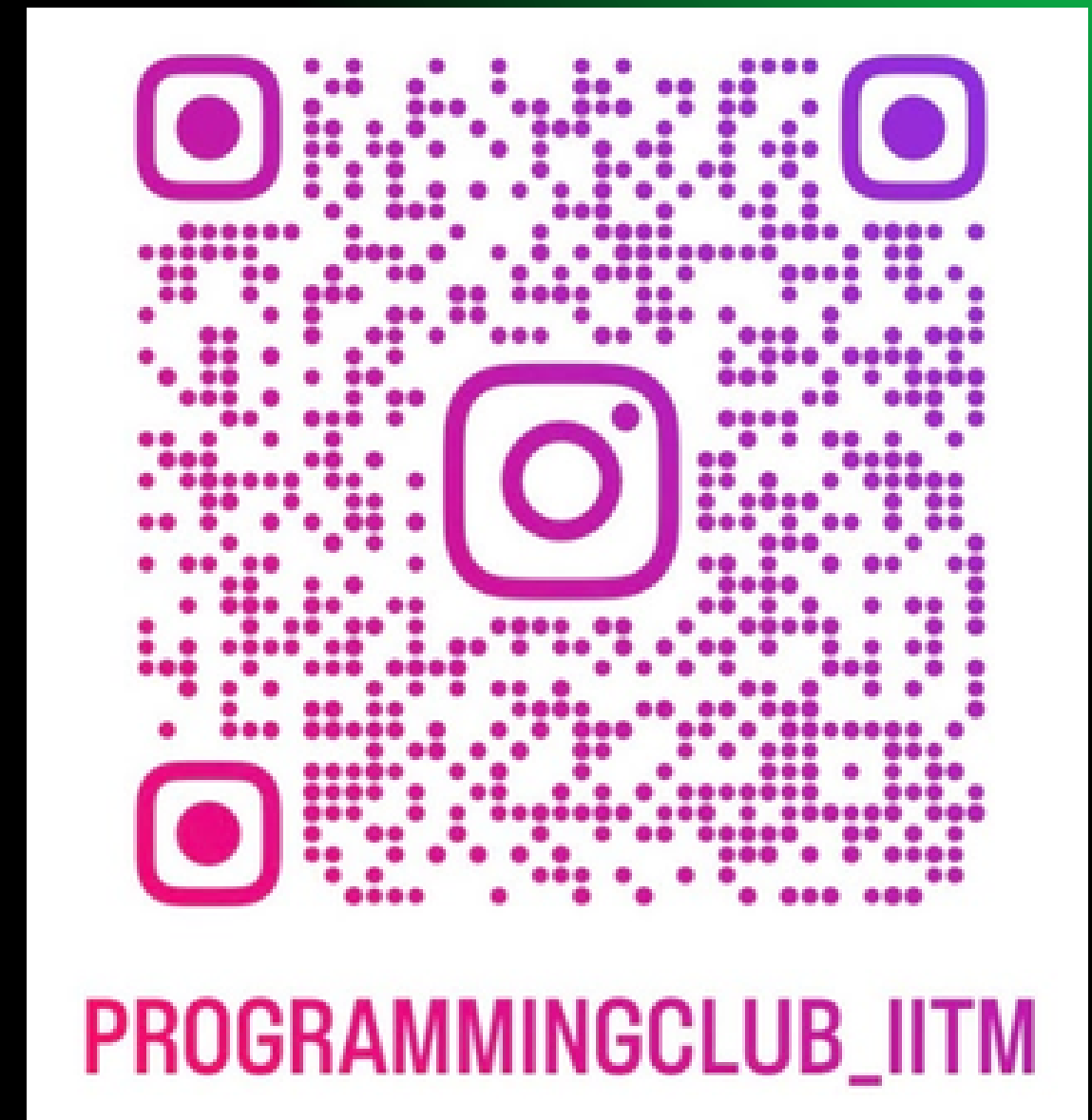


SDS-1

Intro To

Programming



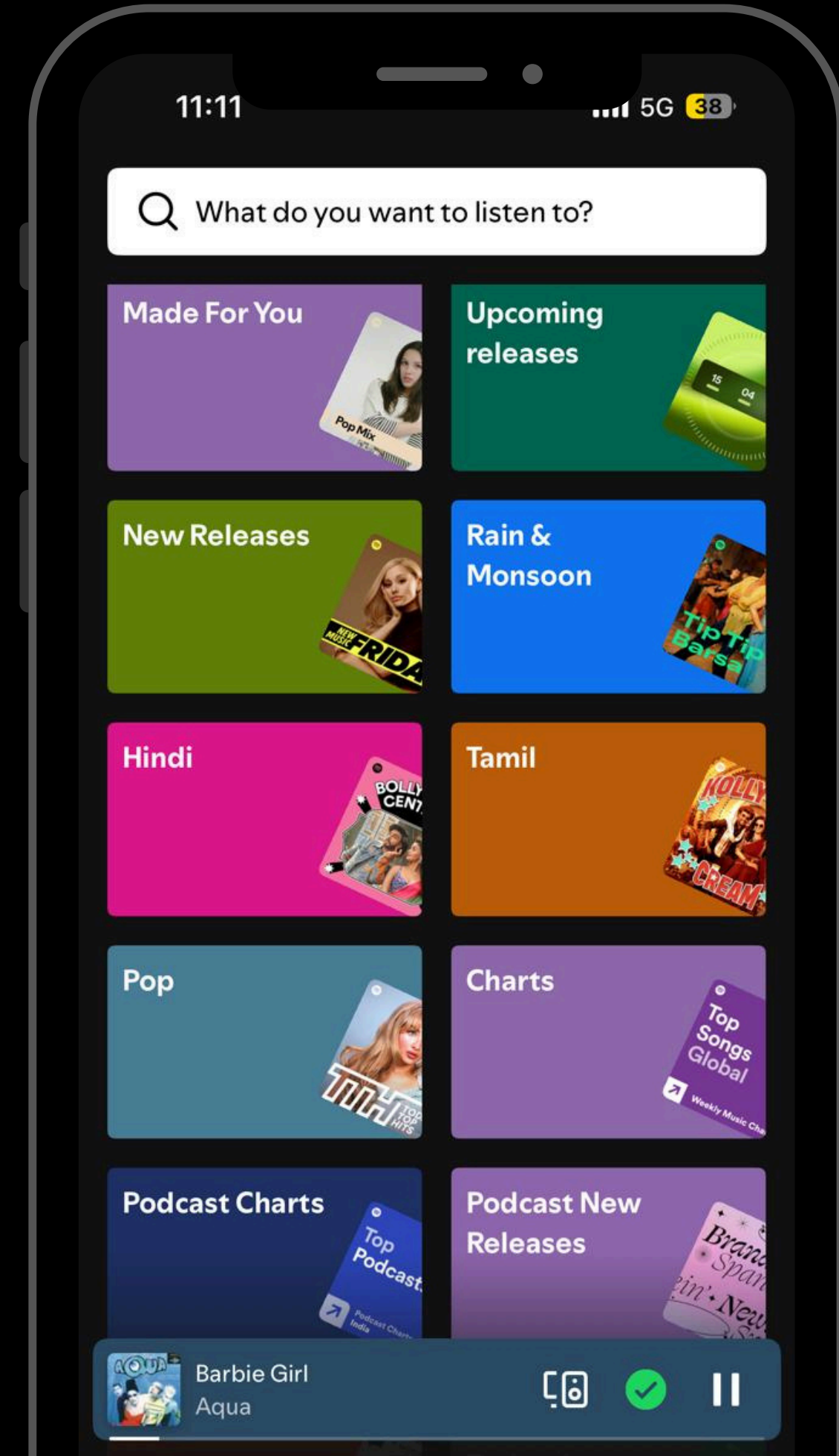


Software Dev

Ragebait

Software Development is writing code that hopefully does what you want.

Reading code, documentation, and finding that one missing semicolon which crashes your entire program are the most important skills you need.



Dev vs CP

	Dev	CP
Goal	Build something useful	Solve problems efficiently
Main Focus	Building & maintaining systems	Usually no permanent product
Important Skills	Frameworks, APIs, databases, Git, UI, system design	Data structures, algorithms, mathematics, optimization
Code	Well documented and large	Short and optimized

How can we start Dev

By learning the most basic concepts known to mankind

Now let's learn some more basic concepts of C++

You're a freshie programmer interested in building your own Spotify playlist. How would you store the songs?

Arrays

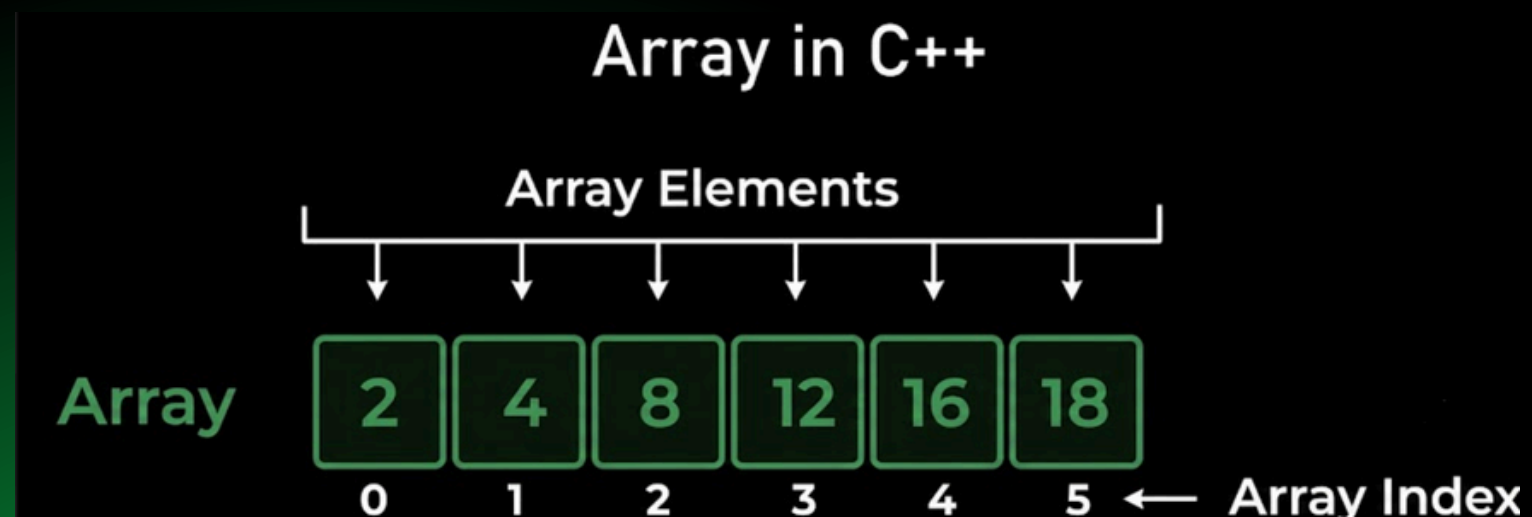
(Fixed Size Containers)

An array stores multiple values of the same data type under one name

Each element is accessed using an index

Indexing starts from 0 in C++

Array size is fixed when the array is created
Arrays are contiguous - elements are beside each other in memory.



```
// Syntax to create an array  
data_type array_name[size];
```

```
// Syntax to access an index i  
array_name[i];
```

What if you don't want a fixed size array, but instead a list that grows and shrinks dynamically?

Vectors

(Dynamic Containers)

A vector is a dynamic array that stores multiple values of the same data type.

Each element is accessed using an index

- Indexing starts from 0 in C++

Unlike arrays, vectors can grow or shrink in size

```
// vector<data_type> vector_name(size)
vector<int> a(5);

// accessing elements
int c=a[0];
```

Vectors

(Dynamic Containers)

```
// 1. Empty vector
vector<int> v1;
// Vector: [ ]

// 2. Vector with initial values
vector<int> v2 = {10, 20, 30, 40};
// Vector: [10, 20, 30, 40]

// 3. Vector with initial size
vector<int> v3(5);
// Vector: [0, 0, 0, 0, 0]

// 4. Vector with initial size and same value
vector<int> v4(5, 10);
// Vector: [10, 10, 10, 10, 10]

// 5. Copy another vector
vector<int> v5 = v2;
// Vector: [10, 20, 30, 40]
```

Vectors

Vectors are part of the C++ Standard Library, so we need to include `<vector>`

```
// Syntax to use vector functions:  
<vector name>.function()
```

Basic Functions:

push_back() — adds an element to the end

pop_back() — removes the last element

size() — returns the number of elements

empty() — checks whether the vector is empty

clear() — removes all elements

front() — accesses the first element

back() — accesses the last element

But wait, how are we actually storing the names in the vector??

Strings

(Dynamic Containers)

A string is a sequence of characters stored together as a single value

Strings can be accessed using an index

Indexing starts from 0 in C++

Strings are dynamic in size and can be modified

```
//Syntax to create a string
std::string string_name="Hello freshers";

// accessing string elements
char c=string_name[0];
```

Strings

```
// Syntax to use string function:  
string_name.function()
```

Basic Functions:

length() / size() — returns the number of characters

empty() — checks whether the string is empty

push_back() — adds a character to the end

pop_back() — removes the last character

substr() — extracts part of a string

find() — searches for a character or substring

append() — adds another string to the end

What if you want the total duration of your playlist, or want to pick a random song?

Functions

(Reusable block of code)

- Performs a specific task within a program
- Helps organize code into smaller, manageable parts
- Can accept inputs (parameters) and return outputs
- Can be called multiple times whenever needed
- **Built-in Functions:**
Predefined functions provided by C++ or its libraries
- **User-defined Functions:**
Functions created by the programmer for a specific purpose

Maintaining three vectors for song name, artist and duration is tedious and error-prone.

Is there any way to associate the three of them with one song?

Structs

- Groups related data together under one name
- Used to create your own custom data type
- Can contain different types of data
- Members are public by default
- Similar to a class, but simpler for basic data grouping

```
// Syntax to create a struct
struct StructName{
    // members (public by default)
};
```

Classes

(Blueprint of an object)

```
// Syntax to create a class
class ClassName{
    private:
        // private members
    public:
        // public members and functions
};
// Syntax to create an object
ClassName objectName;
```

A class is a blueprint for creating objects

An object is an instance of a class that contains its own data and can use the class's functions.

Objects created from a class have their own data.

Members of a Class

Data Members –

Variables that store the object's data

Member Functions –

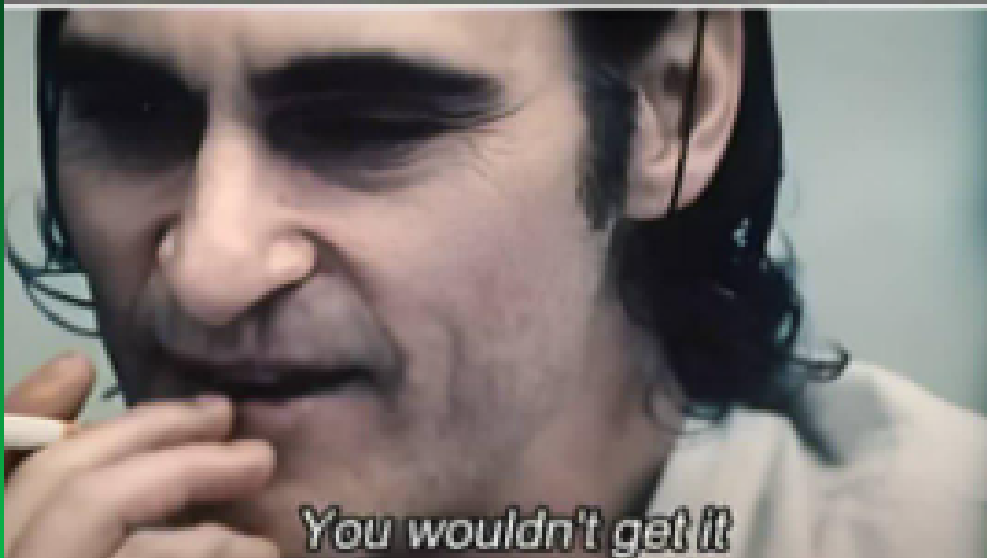
Functions that define the object's behavior

Constructors –

Special functions used to initialize objects

Access Specifiers

```
1 public class Meme
2 {
3     private Joke joke;
4
5     public void setJoke(Joke newJoke)
6     {
7         this.joke = newJoke;
8     }
9 }
```



Keywords that control where the members of a class can be accessed.

Types of Access Specifiers

- **public**
accessible from anywhere
- **private**
accessible only within the class
- **protected**
accessible within the class and its derived classes

Setters & Getters

Used to access and modify private data members from outside the class.

Getter

Function that retrieves or returns the value of a private data member

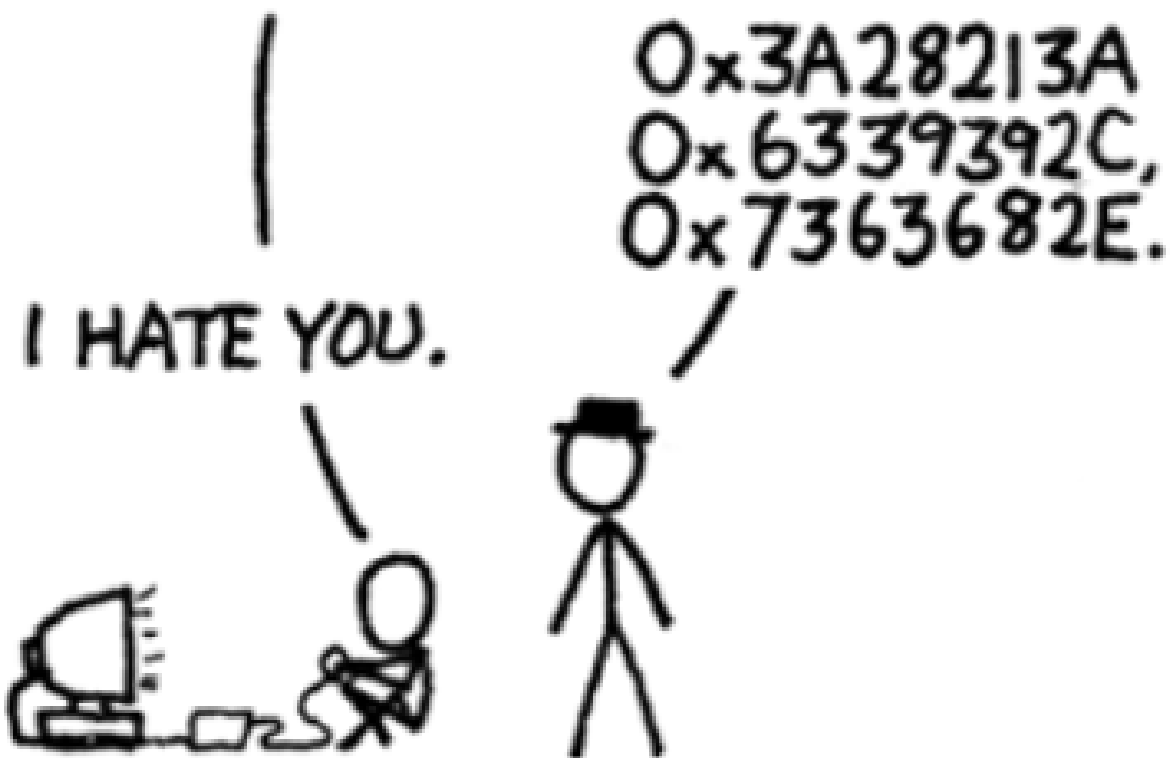
Setter

Function that is commonly used to provide controlled access to private data

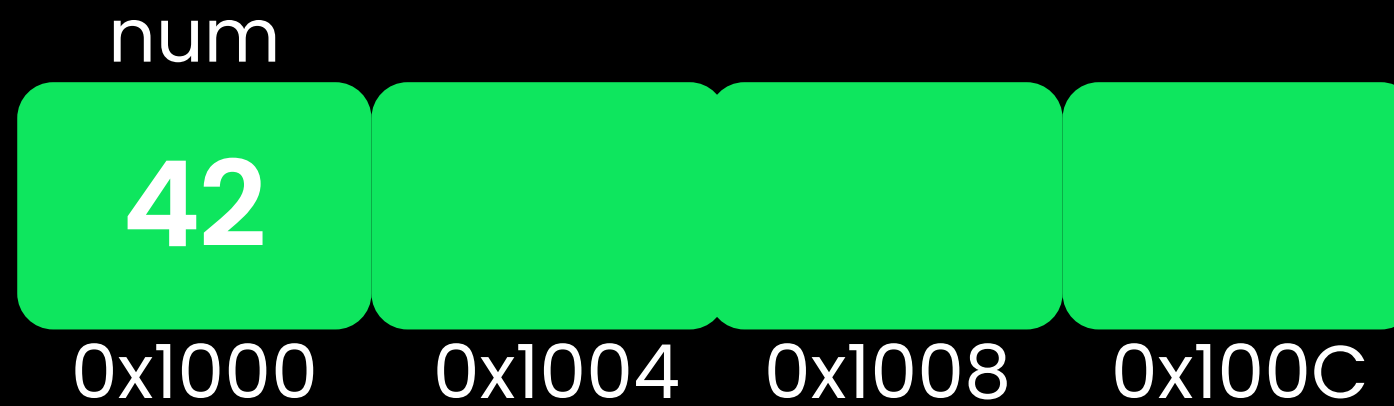
What if your song object had a thousand variables? is there a better way to refer to the song than copying it every time?

Pointers

MAN, I SUCK AT THIS GAME.
CAN YOU GIVE ME
A FEW POINTERS?



- Stores the memory address of another variable
- Can be used to access or modify the value stored at that address
- Can be reassigned to point to different variables
- Can hold a null value (nullptr)
- Declared using the ***** operator
- Syntax:
data_type *pointer_name = &variable;



```
int num = 42;  
int* ptr = &num;  
// *ptr = 42;
```



* “dereferences” the ptr
and fetches the value at
the address it points to.

References

An alias (another name) for an existing variable

Must be initialized when declared

Cannot be reassigned to refer to another variable

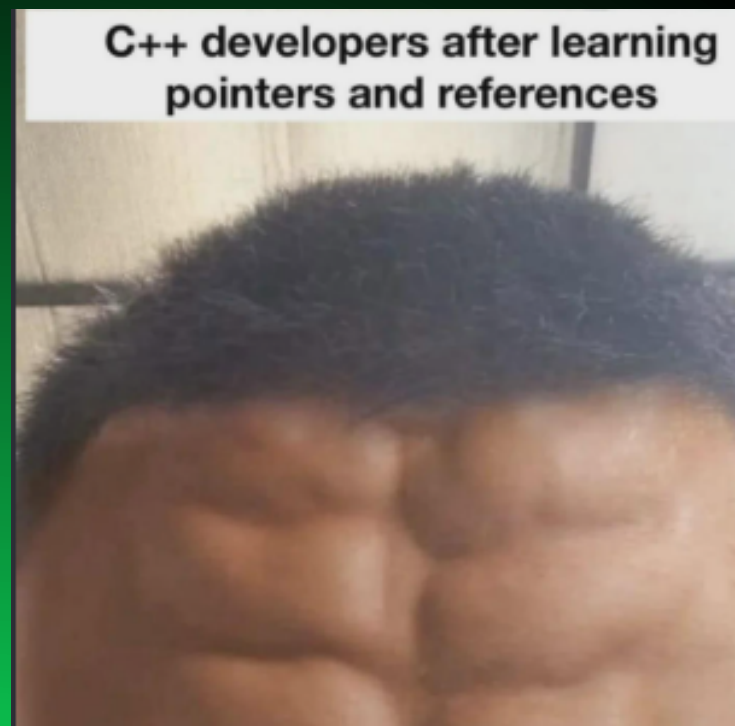
Cannot be null

Declared using the `&` operator

Syntax:

```
data_type &reference_name = variable;
```

Pointers vs References



Pointers	References
Stores a memory address	Acts as an alias
Can be nullptr	Cannot be null
Can use pointer arithmetic	No pointer arithmetic
Access value using *	Used directly like the original variable

Pass by Value

- A copy of the argument is passed to the function
- Changes made inside the function do not affect the original variable
- Suitable when the function only needs a copy of the data
- Can require extra memory for large objects

Pass by Reference

- A reference to the original variable is passed
- Changes made inside the function affect the original variable
- Avoids copying large amounts of data
- Useful when a function needs to modify the original data

Pass by Value VS Pass by Reference

Pass by Value	Pass by Reference
Passes a copy	Passes a reference
Original value unchanged	Original value can be changed
Creates a copy	No copy of the object
Changes stay local	Changes affect the original



Thank You

Programming Club